Movie Predictor

Complete Project Documentation

Version 2.0.0 | FastAPI + React + Machine Learning

This document covers everything about the Movie Predictor project: how it works, what each file does, all the API endpoints, the machine learning models, the React frontend, and how to set it up and run it.

Table of Contents

1. Project Overview

Movie Predictor is a full-stack machine learning web application. It predicts movie ratings, classifies movies as Hit, Average, or Flop, and recommends similar movies. The app uses a synthetic dataset of 5,000 movies and trained ML models.

Key Features

Feature	Description
Random Prediction	Click a button to predict a random movie each time
Rating Prediction	Predict a movie rating on a 1-10 scale using ML
Hit/Flop Classification	Classify a movie as Hit, Average, or Flop
Movie Recommendations	Find similar movies using cosine similarity
Movie Search	Search movies by title with autocomplete
Manual Prediction	Enter custom features to get a prediction

2. System Architecture

The app has three layers: Frontend (React), Backend (FastAPI), and ML Models (scikit-learn). The frontend runs on port 3000 and sends API requests to the backend on port 8000 through a Vite proxy.

How Data Flows

Step	What Happens
1	User clicks "Predict Random Movie" on the frontend
2	React calls GET /api/predict/random via Axios
3	Vite proxy forwards the request to FastAPI on port 8000
4	FastAPI picks a random movie from the 5,000 movie dataset
5	ML Service extracts 31 features and scales them with MinMaxScaler
6	RandomForest model predicts the rating and hit/flop class
7	Response sent back with movie details + prediction + confidence

Architecture Overview

```
FRONTEND (Port 3000) BACKEND (Port 8000)
--------------------- --------------------
React + Vite -----> FastAPI + Uvicorn
Home.jsx /api/ Movies Router (/api/movies)
MovieDetail.jsx proxy Predict Router (/api/predict)
PredictPage.jsx -----> Recommend Router (/api/recommend)
```

```
api.js (Axios) |
ML Service (Singleton)
- rating_model.pkl
- hitflop_model.pkl
- similarity_matrix.npy
```

3. Project Structure

```
movie-predictor/
backend/
app/
main.py -- FastAPI app (entry point)
config.py -- Settings and paths
api/
movies.py -- Movie search/list endpoints
predict.py -- ML prediction endpoints
recommend.py -- Recommendation endpoints
services/
ml_service.py -- ML model loading and inference
recommender.py -- Cosine similarity recommendations
ml/
generate_dataset.py -- Creates 5000 synthetic movies
preprocess.py -- Feature engineering (31 features)
train.py -- Trains all ML models
models/ -- Saved model files (not in git)
data/
tmdb_5000_movies.csv -- Movie dataset (not in git)
requirements.txt
Dockerfile
frontend/
src/
App.jsx -- Router and Navbar
main.jsx -- React entry point
pages/
Home.jsx -- Random predict button
MovieDetail.jsx -- Movie detail + auto predict
PredictPage.jsx -- Manual prediction forms
components/
MovieCard.jsx -- Movie card UI
MovieList.jsx -- Grid of movie cards
SearchBar.jsx -- Search with autocomplete
RecommendationPanel.jsx
RatingPredictor.jsx
HitFlopPredictor.jsx
services/
api.js -- All API calls (Axios)
index.html
vite.config.js
package.json
Dockerfile
docker-compose.yml
start.bat -- Windows quick start
start.sh -- Linux/Mac quick start
```

4. Machine Learning Pipeline

4.1 Data Generation (generate_dataset.py)

This script creates 5,000 fake but realistic movies. Each movie has a budget, revenue, genres, director, cast, rating, and more. Budget and revenue ranges change based on genre (for example, Action movies have bigger budgets than Horror movies).

Detail	Value
Total Movies	5,000 synthetic movies
Genres	18 genres (Action, Drama, Comedy, Horror, etc.)
Directors	15 directors (Nolan, Spielberg, Tarantino, etc.)
Languages	10 languages (mostly English)
Budget Range	$1M to $350M depending on genre
Output File	data/tmdb_5000_movies.csv

4.2 Feature Engineering (preprocess.py)

This script takes the raw movie data and converts it into 31 numeric features that the ML models can use. There are 3 types of features:

Type	Features	Count
Numeric	budget_log, popularity_log, vote_count_log, runtime, is_english, revenue_to_budget, cast_size, release_year, release_month	9
Genre Flags	genre_Action, genre_Comedy, genre_Drama, genre_Horror, genre_Thriller, etc. (one for each of 18 genres)	18
Season Flags	season_Spring, season_Summer, season_Fall, season_Winter	4

Important Transformations

- **budget_log** = log(budget + 1) -- converts large numbers to a smaller scale
- **popularity_log** = log(popularity + 1)
- **revenue_to_budget** = revenue / budget -- how much money the movie made compared to its cost
- **is_english** = 1 if English, 0 otherwise
- **Genre flags** = 1 if the movie belongs to that genre, 0 if not

4.3 Model Training (train.py)

Three models are trained from the processed data:

Rating Prediction Model

Setting	Value
Algorithm	RandomForestRegressor
Number of Trees	200
Max Depth	20
Target	vote_average (rating 1-10)
Scaling	MinMaxScaler (0-1 range)
Train/Test Split	80% train, 20% test
Performance	RMSE = 0.52, R-squared = 0.38

Hit/Flop Classification Model

Setting	Value
Algorithm	RandomForestClassifier
Number of Trees	200
Max Depth	15
Class Weight	balanced (handles uneven class sizes)
Classes	0 = Flop, 1 = Average, 2 = Hit
Split	Stratified 80/20 (keeps class ratios equal)
Performance	Accuracy = 92%

Hit/Flop Definition

Label	ID	Rule
Hit	2	Revenue is more than 3x the budget
Average	1	Revenue is more than the budget (but less than 3x)
Flop	0	Revenue is equal to or less than the budget

Recommendation Engine

- **Method:** TF-IDF vectorizer on combined text (genres + keywords + overview)
- **Max features:** 3,000 words
- **Similarity:** Cosine similarity matrix (5000 x 5000)
- **Output:** Top-N most similar movies with similarity percentages

4.4 Saved Model Files

All trained models are saved in the backend/app/ml/models/ folder. These files are loaded when the backend starts.

File	What It Contains
rating_model.pkl	Trained RandomForest for rating prediction
hitflop_model.pkl	Trained RandomForest for hit/flop classification
preprocessors.pkl	MinMaxScaler + list of 31 feature column names
movie_data.pkl	Full processed DataFrame of 5,000 movies
model_metrics.pkl	Training performance numbers (RMSE, accuracy, etc.)
similarity_matrix.npy	5000 x 5000 cosine similarity matrix
tfidf_vectorizer.pkl	TF-IDF text vectorizer for recommendations

5. Backend - FastAPI

5.1 Entry Point (main.py)

This file creates the FastAPI application, adds CORS middleware (so the frontend can talk to it), and registers 3 API routers.

```
app = FastAPI(title='Movies Prediction App', version='2.0.0')

# Allow frontend to access backend (CORS)
app.add_middleware(CORSMiddleware, allow_origins=['*'])

# Register 3 routers
app.include_router(movies.router) # /api/movies/*
app.include_router(predict.router) # /api/predict/*
app.include_router(recommend.router) # /api/recommend/*

# Run command:
# uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

5.2 Config (config.py)

Setting	Value
APP_TITLE	Movies Prediction App
APP_VERSION	2.0.0
MODELS_DIR	app/ml/models/ (where trained models are stored)
DATA_DIR	backend/data/ (where CSV data is stored)

6. API Reference (All Endpoints)

6.1 Health and Root

Method	Endpoint	Description
GET	/api/health	Health check - returns status, version, app name
GET	/api	Lists all available endpoints

6.2 Movies Endpoints (/api/movies)

Method	Endpoint	Parameters	Description
GET	/movies/search	query (required), limit (default 20)	Search movies by title
GET	/movies/popular	limit (default 20)	Get most popular movies
GET	/movies/top-rated	limit (default 20)	Get highest rated movies
GET	/movies/{movie_id}	movie_id in URL	Get one movie by ID

Example: GET /api/movies/search?query=dark


```
{
"query": "dark",
"count": 3,
"results": [
{
"id": 101,
"title": "Dark Awakening",
"vote_average": 6.7,
"popularity": 172,
"genres_list": ["Fantasy", "Animation"],
"director": "Quentin Tarantino",
"runtime": 95
}
]
}
```


6.3 Prediction Endpoints (/api/predict)

Method	Endpoint	Input	Description
GET	/predict/random	None	Pick random movie and predict
GET	/predict/movie-rating/{id}	movie_id in URL	Predict for a specific movie
POST	/predict/rating	JSON body (31 features)	Predict rating from features
POST	/predict/hitflop	JSON body (31 features)	Classify hit/flop from features
GET	/predict/stats	None	Get model performance metrics

Example: GET /api/predict/random


```
{
"movie": {
"id": 2847, "title": "Final Awakening",
"vote_average": 6.4, "genres_list": ["Crime"],
"director": "Steven Spielberg", "runtime": 144
},
"prediction": {
"movie_id": 2847,
"title": "Final Awakening",
"actual_rating": 6.4,
"predicted_rating": 6.5,
"hitflop_prediction": {
"prediction": "Average",
"confidence": 0.9,
"probabilities": {"flop": 0.05, "average": 0.9, "hit": 0.05}
}
}
}
```


POST Body for /predict/rating and /predict/hitflop

You need to send 31 features as a JSON body. Here are the main ones:

```
{
"budget_log": 18.0,
"popularity_log": 3.0,
"vote_count_log": 5.0,
"runtime": 120.0,
"is_english": 1,
"revenue_to_budget": 2.0,
"cast_size": 5,
"release_year": 2020,
"release_month": 6,
"genre_Action": 1,
"genre_Drama": 0,
... (18 genre flags + 4 season flags)
}
```

6.4 Recommendation Endpoints (/api/recommend)

Method	Endpoint	Parameters	Description
GET	/recommend/similar/{id}	n (default 10)	Get N similar movies
GET	/recommend/by-genre	genre, n, min_rating	Recommend by genre

Example: GET /api/recommend/similar/101?n=3


```
{
"movie_id": 101,
"title": "Dark Awakening",
"recommendations": [
{
"id": 456, "title": "Lost Kingdom",
"similarity_score": 0.87,
"vote_average": 7.1,
"director": "Jordan Peele"
```

```
}
]
}
```

7. Frontend - React + Vite

7.1 Libraries Used

Library	Version	What It Does
React	18.3	User interface framework
React Router DOM	6.26	Page navigation (client-side routing)
Axios	1.7	Makes HTTP requests to the backend API
Vite	5.4	Development server and build tool (very fast)

7.2 Vite Proxy Setup (vite.config.js)

Vite dev server runs on port 3000. Any request starting with /api is automatically forwarded to the backend on port 8000. This way the frontend does not need to know the backend URL.

```
server: {
port: 3000,
proxy: {
'/api': {
target: 'http://localhost:8000',
changeOrigin: true
}
}
}
```

7.3 Routes (App.jsx)

The app has 3 pages. A navbar at the top lets users switch between Home and Predict pages.

URL Path	Page Component	What It Shows
/	Home	Random predict button + prediction result
/movie/:id	MovieDetail	Movie details + auto prediction + recommendations
/predict	PredictPage	Manual prediction forms (rating + hit/flop)

7.4 API Client (api.js)

All backend API calls are defined in one file using Axios. The base URL is /api and the timeout is 10 seconds.

Function Name	HTTP Method	Endpoint
healthCheck()	GET	/health
searchMovies(query, limit)	GET	/movies/search
getPopularMovies(limit)	GET	/movies/popular

Function Name	HTTP Method	Endpoint
getTopRatedMovies(limit)	GET	/movies/top-rated
getMovieById(id)	GET	/movies/{id}
predictRandomMovie()	GET	/predict/random
predictRating(features)	POST	/predict/rating
predictHitFlop(features)	POST	/predict/hitflop
predictMovieRating(movieId)	GET	/predict/movie-rating/{id}
getModelStats()	GET	/predict/stats
getSimilarMovies(movieId, n)	GET	/recommend/similar/{id}
recommendByGenre(genre, n, min)	GET	/recommend/by-genre

8. React Components

Pages

Home.jsx - Landing Page

Shows a big 'Predict Random Movie' button. When you click it, the app picks a random movie from the database, runs the ML model, and shows the movie details with the AI prediction (rating + hit/flop + confidence). Every click gives a different movie.

- **API Call:** GET /api/predict/random
- **Shows:** Movie title, tagline, genres, director, runtime, year, overview
- **Prediction Card:** Predicted rating, actual rating, hit/flop badge, confidence %

MovieDetail.jsx - Movie Detail Page

When you click on any movie card, you come here. It automatically fetches the movie data AND runs the prediction at the same time (parallel API calls). Also has a button to get similar movie recommendations.

- **API Calls:** getMovieById(id) + predictMovieRating(id) in parallel
- **Extra:** 'Get Similar Recommendations' button calls getSimilarMovies(id)

PredictPage.jsx - Manual Prediction

Two forms side by side: Rating Predictor (left) and Hit/Flop Predictor (right). You enter movie features like budget, runtime, genre, etc. and get a prediction.

Reusable Components

Component	Props	What It Does
MovieCard	movie, onPredict	Clickable card with title, genre tags, rating, popularity, director. Click goes to /movie/{id}. Badge: Hit/Avg/Flop
MovieList	movies, title, loading, emptyMessage	Responsive grid of MovieCards. Auto-fill columns (min 280px each)
SearchBar	onSelect	Debounced search (300ms delay). Dropdown with results. Click result goes to movie detail
RecommendationPanel	recommendations, sourceMovie, loading	"Because you liked [movie]" header + list of similar movies with similarity score %
RatingPredictor	None	Form with 9 number inputs + 18 genre toggles. Season auto-calculated. Shows predicted rating
HitFlopPredictor	None	Same form as RatingPredictor but shows Hit/Avg/Flop label with emoji and per-class probabilities

9. Backend Services

9.1 MLService (ml_service.py) - Singleton

This is the core ML service. It loads all model files when the app starts and provides methods for prediction and search. It is a singleton, meaning only one instance exists in the entire app.

How It Loads Models

On startup, it loads 6 files from the ml/models/ directory: rating model, hitflop model, preprocessors (scaler + feature columns), movie data, metrics, and similarity matrix.

Methods

Method	Input	Output	What It Does
predict_rating(features)	dict	float (1-10)	Scale features -> predict -> clip to 1-10 -> round
predict_hitflop(features)	dict	dict	Scale -> predict -> return label + confidence + probabilities
_prepare_features(features)	dict	numpy array	Map dict to 31 columns -> fill missing with 0 -> MinMaxScale
search_movies(query, limit)	str, int	list of dicts	Case-insensitive title search in the DataFrame
get_movie_by_id(movie_id)	int	dict or None	Find movie by ID column
get_model_metrics()	None	dict	Return saved training metrics

9.2 RecommenderService (recommender.py) - Singleton

This service handles movie recommendations using the pre-computed cosine similarity matrix. It finds the most similar movies based on text features (genres, keywords, overview).

Method	Input	Output	What It Does
recommend(movie_id, n)	int, int	list of dicts	Find top-N similar movies with similarity scores
recommend_by_features()	dict, int	empty list	Not implemented yet (placeholder)

10. Docker and Deployment

docker-compose.yml

Two services are defined: backend and frontend. The backend builds from backend/Dockerfile and exposes port 8000. The frontend builds from frontend/Dockerfile and exposes port 3000 (mapped to port 80 inside the container).

```
services:
backend:
build: ./backend
ports: 8000:8000
volumes: data/ and models/ mounted
restart: unless-stopped

frontend:
build: ./frontend
ports: 3000:80
depends_on: backend
restart: unless-stopped
```

11. How to Setup and Run

Option 1: Windows Quick Start

```
cd movies-prediction-app
start.bat
```

This script trains models (if needed), starts the backend, and starts the frontend in separate windows.

Option 2: Manual Setup

Step 1: Install Backend Dependencies

```
cd backend
pip install -r requirements.txt
```

Step 2: Generate Data and Train Models

```
python -m app.ml.generate_dataset
python -m app.ml.train
```

Step 3: Start Backend Server

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Step 4: Start Frontend (in a new terminal)

```
cd frontend
npm install
npm run dev
```

Frontend will be at http://localhost:3000 and backend at http://localhost:8000

Option 3: Docker

```
docker-compose up --build
```

12. Tech Stack

Layer	Technology	Version	Purpose
Frontend	React	18.3	User interface
Frontend	Vite	5.4	Dev server and bundler
Frontend	React Router DOM	6.26	Page navigation
Frontend	Axios	1.7	HTTP requests
Backend	FastAPI	0.115	REST API framework
Backend	Uvicorn	0.30	ASGI server
Backend	Pydantic	2.9	Data validation
ML	scikit-learn	1.5	RandomForest models + TF-IDF
ML	pandas	2.2	Data processing
ML	numpy	1.26	Numerical computation
ML	joblib	1.4	Model saving/loading
DevOps	Docker Compose	3.8	Container orchestration